Order Book: Rules & Design

Oct 21, 2026 · @Manu

Overview

A fair order book follows one principle: **the best price trades first, and among orders at the same price, whoever arrived first trades first.** Everything else in this document either supports that principle or covers an edge case around it.

An order book holds everyone's resting buy and sell orders for one asset. When someone "buys at the current ask" or "sells at the current bid", they are a taker crossing the spread against a maker's resting order.

Term	Meaning
Bid	A buy order: buy up to Q units at price P or lower
Ask (offer)	A sell order: sell up to Q units at price P or higher
Best bid / best ask	The highest bid and the lowest ask currently in the book
Spread	Best ask minus best bid
Price level	All resting orders on one side at one price
Resting order	A limit order sitting in the book, waiting to be matched
Maker	Owner of a resting order; they add liquidity
Taker (aggressor)	Owner of an incoming order that trades immediately; they remove liquidity
Tick	The smallest allowed price step
Lot	The smallest allowed quantity step

Data model rules

The book is two sorted sides of price levels, and each price level is a first-in, first-out queue of orders.

1. **Two sides, sorted by priority.** Bids run from highest price to lowest; asks run from lowest to highest. The first level on each side is the best price.

2. **Each price level is a queue.** New orders join the back; fills come from the front.
3. **Every order carries these fields:**
    - Order ID, unique and assigned by the engine
    - Owner (account) ID
    - Side (buy or sell) and type (limit, market, IOC and so on)
    - Limit price, in ticks
    - Original quantity and remaining quantity
    - Sequence number, assigned by the engine on arrival
    - Status: accepted, partially filled, filled, cancelled or rejected
4. **Integers only.** Store prices as a count of ticks (100.50 becomes 10050) and quantities in the smallest unit. Floating-point rounding will eventually produce a wrong fill.
5. **Enforce the grid.** Reject any price that isn't a multiple of the tick size and any quantity that isn't a multiple of the lot size.
6. **One book per instrument.** Two assets means two independent books.
7. **Only the matching engine changes the book.** Nothing else adds, removes or edits orders directly.

Matching rules

Use price-time priority: match the best price first, then the earliest order at that price, and always trade at the resting order's price.

1. **Price priority.** A better price always beats an earlier time. A bid at 101 is filled before a bid at 100, however long the 100 bid has waited.
2. **Time priority.** At the same price, the order with the lowest sequence number goes first. The engine assigns that number on arrival; never trust a client timestamp.
3. **One event at a time.** Process each new order, cancel or amendment completely before starting the next. The same sequence of inputs must always produce the same trades.
4. **When an order crosses.** An incoming buy matches if its price is at or above the best ask. An incoming sell matches if its price is at or below the best bid.
5. **Trade at the resting price.** The maker set that price first, so the trade happens there. If the taker was willing to pay more, the taker keeps the difference.
6. **Walk the book.** Fill against the front order of the best level. When that level is empty, move to the next level. Stop when the incoming order is fully filled or no longer crosses.
7. **Handle the leftover by order type.** A limit order's remainder rests in the book at its limit price. A market or IOC remainder is cancelled.
8. **Record every fill as a trade.** Each trade needs a trade ID, price, quantity, maker order ID, taker order ID, aggressor side and sequence number.

9. **The book is never crossed at rest.** After every event, the best bid must be strictly below the best ask. If this ever fails, there is a bug.

Worked example

Starting book, with sequence numbers showing arrival order:

Side	Price	Qty	Owner	Seq
Ask	102	10	Carol	3
Ask	101	3	Bob	2
Ask	101	5	Alice	1
Bid	99	4	Dan	4
Bid	98	6	Eve	5

Frank sends a buy limit for 10 at 101 (seq 7):

Step	Matched against	Qty	Price	Why
1	Alice	5	101	Best ask, and earliest at that price
2	Bob	3	101	Same price, next in the queue
3	None	2 rest	101	Carol's 102 is above Frank's limit, so the remainder becomes a bid

After this, the best bid is 101 (Frank, 2) and the best ask is 102 (Carol, 10). The book is not crossed.

If Frank had instead sent a buy limit for 10 at 105, he would fill 5 from Alice at 101, 3 from Bob at 101 and 2 from Carol at 102. He pays the makers' prices, not 105.

Same price: who gets filled first?

Not random. Fill the maker whose order reached the engine first (FIFO). This is the standard rule on most stock and crypto exchanges, and it is also the cheapest rule to implement.

In the worked example above, Alice and Bob both ask 101. Alice arrived first (seq 1), so a buyer fills Alice completely before Bob receives anything.

Why FIFO is the fair choice

- **It rewards commitment.** The first maker took the risk of quoting that price before anyone else did. Being first in line is their reward.
- **It is predictable.** Each maker can know their place in the queue and how much volume stands ahead of them.
- **It is verifiable.** Replaying the event log produces identical fills, so anyone can audit that the rule was followed.
- **It is unambiguous.** Sequence numbers are unique, so there are never true ties.

Why not random

- **Nobody can check it.** Participants must trust your random number generator, and you cannot easily prove you didn't favour someone.
- **It is not reproducible** unless you log every random seed.
- **It rewards splitting orders.** Ten orders of 1 unit get ten lottery tickets where one order of 10 gets one. The book fills with tiny orders.
- **It weakens the incentive to quote early.** If arriving first earns nothing, makers wait, and spreads widen.

The alternatives

Rule	How a fill at one price is split	Where it appears	Trade-off
FIFO (time priority)	The earliest order is filled in full, then the next	Most stock and crypto exchanges	Rewards being early; constant cost per fill
Pro-rata	In proportion to each order's size	Some interest-rate futures markets	Rewards large orders and invites overstated sizes; every fill touches every order at the level and needs a rounding rule
Hybrid	The first order gets a priority share; the rest is split pro-rata	Some futures exchanges	Harder to explain, implement and test
Random	A lottery among orders at the level	Rare in practice	Unverifiable, non-deterministic, encourages order splitting

Recommendation: implement FIFO. If you want to experiment later, design the allocation step so it can be swapped for another policy without touching the rest of the engine.

One rule protects FIFO from abuse: an order that grows in size or changes price loses its place in the queue (see Cancels and amendments).

Order types

Start with limit and market orders, and add the others once matching is solid and tested.

Type	Behaviour	Leftover quantity	Edge cases to decide
Limit	Trades up to its limit price	Rests in the book	None beyond the matching rules
Market	Takes the best available prices, whatever they are	Cancelled; a market order never rests	Empty book: reject. Thin book: fill what exists, cancel the rest. Add a price collar so it cannot sweep absurdly far
IOC (immediate-or-cancel)	A limit order that never rests	Cancelled	A zero fill is a valid result; report it clearly
FOK (fill-or-kill)	Fills completely and immediately, or not at all	The whole order is cancelled	Check available quantity up to the limit price before touching the book, so nothing partially fills
Post-only	Must rest; never takes liquidity	Rests in the book	If it would cross on arrival, reject it (simplest) or reprice it one tick away from the opposite best. Pick one and document it

Market buys need a cap. A buy with no price limit has no maximum cost, so you can't reserve funds for it. Either require a maximum spend, or convert it to a limit order at the collar price.

Time in force (optional). GTC (good-till-cancelled) orders stay until cancelled. Day orders are cancelled automatically at the end of a session.

Cancels and amendments

An amended order keeps its queue position only when its quantity goes down; any other change sends it to the back of the queue.

Change	Keeps queue position?	Why
Reduce quantity	Yes	It gives up part of its claim and harms nobody behind it
Increase quantity	No: new sequence number, back of the queue	Otherwise anyone could hold an early spot with 1 unit, then grow it to 1,000
Change price	No: treat as cancel plus a new order	It is a new commitment at a new level, and it may cross and trade immediately
Reduce to zero or below the filled amount	Treat as a cancel	Nothing valid is left to rest

- **Cancelling is immediate.** Once processed, a cancelled order cannot fill.
- **Partial fills stand.** Cancelling a partially filled order cancels only the remaining quantity.
- **Cancel races.** If a cancel arrives after the order has fully filled, reject it as "too late to cancel". Never undo a trade.
- **Only the owner can cancel or amend** their own orders.
- **Market-maker tools (optional).** Mass cancel removes all of an account's orders in one event. Cancel-on-disconnect removes them automatically if the connection drops.

Self-trade prevention

Block any trade where the same owner is on both sides. Without this rule, people can fake trading volume (wash trading). Pick one policy and apply it to everyone.

Policy	What happens when a taker reaches its own resting order
Cancel newest	The incoming order's remainder is cancelled; matching stops
Cancel oldest	The resting order is cancelled; matching continues with the next order in the queue

Policy	What happens when a taker reaches its own resting order
Cancel both	Both orders are cancelled
Decrement	Both orders are reduced by the smaller quantity, and no trade is recorded

Decide what "same owner" means: the same account, or any account belonging to the same firm. Run the check inside the walk-the-book loop, just before each fill.

Validation and balances

Reject bad orders before they touch the book, and lock funds when an order is accepted, not when it fills.

Reject an order when

- Its quantity is zero, negative or not a multiple of the lot size
- Its price is zero, negative or not on the tick grid
- Its price is too far from a reference price such as the last trade (a price band that catches "fat finger" mistakes)
- The instrument or account is unknown, or the book is closed or halted
- Its client order ID duplicates one that is still live
- The account lacks the available balance to cover it

Never silently fix an order, for example by rounding its price. Reject it, with a reason code the sender can act on.

Reserve balances on acceptance

- **A buy limit** reserves price × quantity of the quote currency.
- **A sell** reserves the quantity of the asset being sold.
- **On a fill,** move the reserved amount to the counterparty.
- **When a buy fills below its limit,** release the unused difference back to available.
- **On cancel,** release whatever is still reserved.
- **Available balance never goes below zero.** This is what stops someone from placing sell orders for 10 units while owning 5.

Fair information

Everyone sees the same market data at the same time, and nobody sees an order before it is in the book.

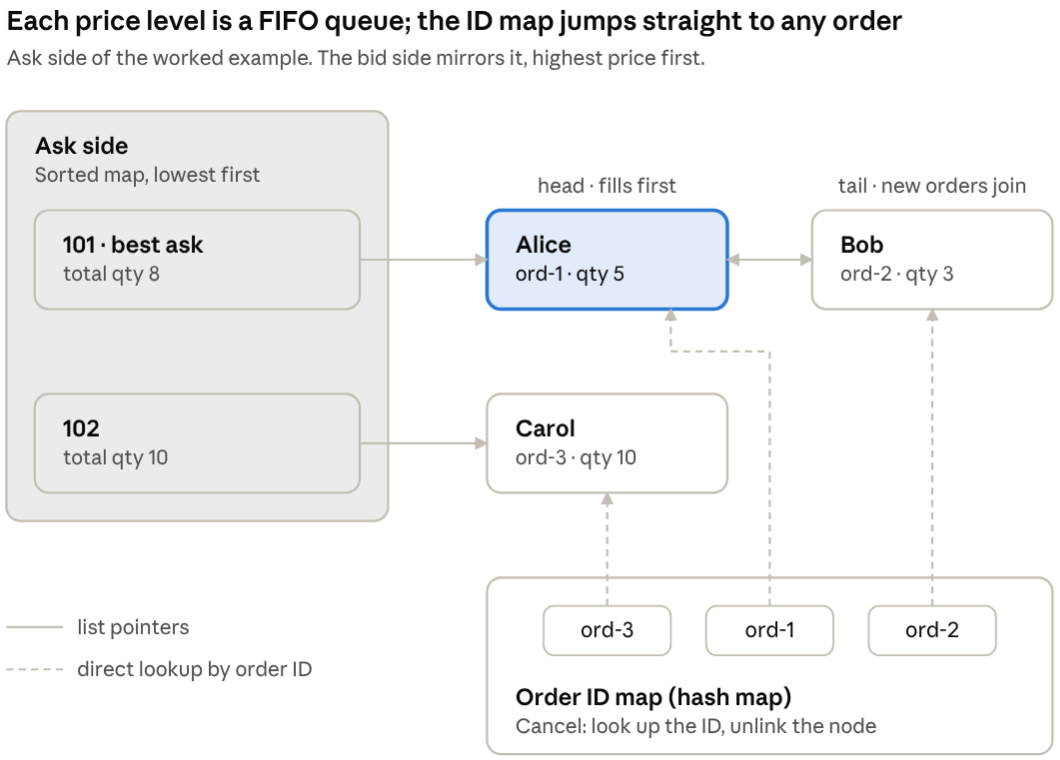
- **Publish after every event, in order.** Tag each update with its sequence number so clients can detect a missed message.
- **Choose your depth of data.** Level 1 is best bid and ask only. Level 2 is total quantity per price level. Level 3 is every individual order, with anonymised IDs.
- **Keep the book anonymous.** Never reveal who owns which order.
- **No peeking.** No one, including whoever runs the engine, may see or act on an incoming order before it is processed. Doing so is front-running.
- **Separate private and public messages.** Order acknowledgements and fill reports go only to the owner. Trade prints and book updates go to everyone.
- **Publish the fee schedule.** If makers earn a rebate and takers pay a fee, write it down and apply it equally to everyone in the same tier.

Designing an efficient OrderBook class

FIFO is also the fast choice. With four structures, each fill and each cancel costs constant time, and only creating or removing a whole price level depends on how many levels exist.

The four structures

1. **A sorted map of price levels per side.** Key it by price, using a balanced tree or equivalent, so the best price sits at one end. If prices stay in a known, bounded range, an array indexed by tick plus a cached best-price position is even faster.
2. **A doubly linked list of orders inside each level.** The head is the oldest order. Keep a running total of the level's quantity alongside it, for Level 2 data and FOK checks.
3. **A hash map from order ID to that order's list node.** A cancel jumps straight to the node and unlinks it, with no scanning.
4. **A hash map from price to level** (optional). Adding to an existing level then skips the tree search.



ask side · 2 price levels, 3 orders, ID map

Each level points at the head of its queue. Fills take from the head, new orders join the tail, and the ID map lets a cancel reach any order without scanning.

Cost targets (L = number of price levels)

Operation	Target cost
Read best bid or best ask	Constant
Add an order at an existing price	Constant with the price map; log L without it
Add an order at a new price	log L
One fill against the front of a level	Constant
Cancel by order ID	Constant, plus log L if the level becomes empty
FOK pre-check	Proportional to the levels it inspects

Pro-rata and random allocation cost more. Each fill has to visit every order at the level, not just the front one.

Practical rules

- **Remove a level as soon as it is empty.** Otherwise the best price points at a level with nothing in it.
- **Keep the core single-threaded.** Put incoming events into one queue and process them in order. Concurrency belongs at the edges (network, market data), not inside matching.
- **Return events from every call instead of printing.** Each call should return what happened: accepted, rejected, trade, rested, cancelled. This makes testing and publishing straightforward.
- **Keep separate jobs separate.** The OrderBook class can own the book and matching. Balances and risk checks, the event log and market data publishing work better as separate components that the engine calls or feeds.

Invariants and test checklist

Assert these invariants after every event in your tests. Most order book bugs break at least one of them.

Invariants

- [ ] Best bid is strictly below best ask whenever both exist
- [ ] For every order, filled + remaining + cancelled = original quantity
- [ ] Every trade has equal buy and sell quantity, at a price within both parties' limits
- [ ] Every trade's price equals the resting order's price
- [ ] Within a price level, fills happen in sequence-number order
- [ ] Each level's total quantity equals the sum of its orders' remaining quantities
- [ ] Every order in the ID map is in exactly one level, and every order in a level is in the ID map
- [ ] No empty price levels remain in the book
- [ ] Replaying the event log rebuilds an identical book and identical trades
- [ ] For every account, reserved + available = total, and nothing is negative

Scenarios to test

- [ ] Market order against an empty book
- [ ] Limit order exactly at the best opposite price
- [ ] Large order that sweeps several levels
- [ ] Two makers at the same price: the earlier one fills first

- [ ] Amend down keeps queue position; amend up and price change lose it
- [ ] Cancel that arrives after a full fill is rejected
- [ ] Partial fill, then cancel of the remainder
- [ ] Self-trade under your chosen policy
- [ ] FOK one unit short of full fill: no trades at all
- [ ] Post-only order that would cross on arrival
- [ ] Buy filled below its limit releases the unused reserved funds

Going further: batch auctions

Continuous FIFO matching rewards speed, because whoever reaches the engine first wins the queue. Frequent batch auctions remove that race. Build the continuous book first, then consider this as a second mode.

- **How it works.** Collect orders over a short interval, such as 100 milliseconds. Then compute one clearing price, the price at which the most volume can trade, and fill every crossing order at that single price.
- **You still need a tie rule.** When more volume is offered at the clearing price than can trade, decide how to split it, for example by time or pro-rata.
- **Trade-offs.** Orders wait up to one interval, the logic is more complex, and traders may find it less familiar.
- **Where it already appears.** Many exchanges use a single auction like this for their opening and closing prices.

The idea comes from Budish, Cramton and Shim's 2015 paper on the high-frequency trading arms race, if you want to read the economics behind it.